



Especificación de requisitos de software de “TicketAPI Poncho”

Información general del documento

Campo	Información
Proyecto	TicketAPI Poncho
Tipo de Sistema	API CRUD
Versión del Documento	5.0
Fecha	06/06/2026
Equipo de Trabajo	Arce Lautaro Martín Gallo Pablo Javier

1. Introducción:

La Fiesta del Poncho, celebrada anualmente en Catamarca, es el festival cultural y folclórico más importante de la región. Durante su celebración, el Predio Ferial Catamarca tiene miles de asistentes a sus espectáculos y recitales.



En este contexto de alta concurrencia y demanda, la gestión eficiente, segura y escalable de la venta de entradas se vuelve un pilar crítico para la organización de los espectáculos. El presente proyecto surge bajo la inspiración directa de esta festividad, buscando resolver las complejidades asociadas a la distribución de tickets para sus espectáculos y recitales.

1.1 Propósito del sistema:

El propósito de este proyecto es el diseño y desarrollo de "TicketAPI Poncho", un Sistema Backend basado en una Arquitectura de API REST mediante el uso de Django Rest Framework (DRF). El sistema tiene como objetivo centralizar la gestión de eventos, la segmentación de sectores en el Predio Ferial y el procesamiento transaccional de compras de entradas. El sistema tiene un motor de precios dinámicos (modelos de polinómico determinista). Esto automatiza el ajuste del valor de las entradas según la velocidad de la demanda y la ocupación de los sectores.

Con esto, el sistema garantiza la integridad de datos, el control de la sobreventa y un acceso seguro mediante mecanismos de autenticación y autorización.

Aparte cuenta con un módulo de WebSocket para la transferencia de gráficas para medición de datos a tiempo real.

1.2 Alcance del sistema:

El sistema es exclusivamente un núcleo lógico de backend y la arquitectura de API REST y WebSocket. El alcance del proyecto viene desde la persistencia de datos en una base de datos relacional (noches de festival, sectores del predio ferial y transacciones) hasta los endpoints seguros.

- Dentro del desarrollo de este laboratorio no se tienen interfaces de usuario frontend. Todo el proyecto está concentrado en la validación, seguridad y procesamiento matemático de los datos en el servidor.



- Por restricciones de tiempo y entendiendo esto como un laboratorio, decidimos no acoplar una nueva entidad llamada “Instalacion”, propietaria de varios “SectorEntrada” que se conecta con “Evento” siendo un intermedio entre ellos y permitiendo más de un único predio en el caso de una futura expansión del festival.

1.3 Descripción general y alcance:

1.3.1 Perspectiva del producto:

TicketAPI Poncho no es una aplicación aislada, sino que funciona como componente orquestador de un sistema de eventos. Este sistema interactúa con:

1. Una base de datos relacional para el control de stock.
2. Un servicio externo de conectividad financiera y cambiaria (la API de cotización del mercado cambiario)
3. Un cliente externo autorizado, como aplicaciones móviles de escaneo de entradas en los accesos al predio ferial o páginas web de venta, los cuales consumen los servicios del sistema en formato JSON.

1.3.2 Funciones del producto:

El sistema organiza sus capacidades en los siguientes módulos:

- Gestión de identidad y seguridad (JWT): Controla el ingreso al sistema emitiendo tokens de acceso seguro de corta duración.
- Control de acceso basado en roles: Segmenta qué capacidades da el sistema según el tipo de usuario por medio de permisos de Django.



- Catálogo dinámico de la cartelera: Da las ofertas de espectáculos disponibles para el festival, permite búsqueda con filtros de ordenamiento y entrega páginas de datos.
- Motor de checkout: Procesa solicitudes de compra de entradas de manera sincrónica para asegurar no vender más entradas de las permitidas por el predio.
- Motor de precios dinámicos: Analiza con un modelo polinomial determinista el ritmo de venta del festival, junto a otros datos relevantes, dando variaciones de precio en caliente para una mejor eficiencia de la organización.
- Seguimiento de datos a tiempo real a través de gráficos globales y específicos.

1.3.3 Características de usuarios:

El sistema interactúa de manera diferente con 3 perfiles de usuario:

- Comprador: El interesado en asistir al evento. No necesita conocimientos técnicos. Solo se limita a autenticarse, consultar cartelera y realizar la compra de entradas.
- Productor: El personal administrativo del festival o de secretarías de cultura. Debe tener conocimientos operativos del negocio. Debe dar alta a los artistas, definir la capacidad de las plateas y monitorear el comportamiento del motor de precios dinámico.
- Soporte: El perfil de altos conocimientos técnicos encargado del sistema, su mantenimiento en conexiones externas, gestión de roles y auditoría del estado de la plataforma.

1.3.4 Limitaciones y restricciones:

El sistema está condicionado por las siguientes limitaciones:

- Dependencia de terceros: Los precios en dólares dados al usuario están directamente en función de la disponibilidad del servicio



externo dolarapi.com. El coeficiente de popularidad de los artistas se consigue a través del consumo de la API de YouTube.

- Restricciones de infraestructura física: El sistema no tiene stock infinito, el stock está dado por las capacidades del predio que están establecidos por normativas civiles.
 - Regulación antifraude y reventas: Por políticas comerciales de control de eventos, el sistema tiene restringido el volumen de venta de tickets por cuenta, para mitigar las reventas.
 - Restricciones de cátedra: El diseño únicamente puede ser hecho en las herramientas de la cátedra de desarrollo de APIs (Django Rest Framework, SQLite y JWT).
-

2. Referencias:

- IEEE Std 830-1998 / IEEE Std 29148-2018 (Especificación de Requisitos de Software): Estándar internacional utilizado para la estructura, redacción y organización de este documento (ERS) (viene de la cátedra de ingeniería en software 1)
- Estándar IEEE 1016-2009 (Descripción de Diseño de Software - SDD): Norma técnica de referencia utilizada para el modelado de la arquitectura del sistema (viene de la cátedra de ingeniería de software 2)
- YouTube Data API v3 - servicio de información referente a YouTube sobre canales, métricas, etc.
- DolarApi - Servicio de Cotización en Línea: Documentación y especificación técnica de la API REST expuesta en [DolarApi](#).
- Ley Nacional N° 24.240 de Defensa del Consumidor (República Argentina): Marco regulatorio que rige sobre la transparencia en los precios publicados.
- Código Contravencional de la Provincia de Catamarca (Ley N° 5.171 / Modificaciones vigentes): Normativa legal específica que sanciona el "lucro indebido con entradas de espectáculos públicos".



3. Requerimientos específicos:

3.1 Requerimientos funcionales:

RF-01: Gestión de eventos. El sistema debe permitir a los usuarios administradores el uso del CRUD (Create, Read, Update and Delete) de las distintas noches del Poncho detallando nombre, fecha, hora, lugar y artista principal.

RF-02: Gestión de sectores. El sistema debe permitir a los usuarios administradores el uso del CRUD (Create, Read, Update and Delete) de los sectores específicos del predio detallando nombre, evento asociado, capacidad máxima, entradas vendidas y precio base en pesos.

RF-03: Gestión de usuarios. El sistema debe permitir a los usuarios el uso del CRUD (Create, Read, Update and Delete) de sus propios usuarios, o en el caso de un usuario administrador de otros usuarios incluso, permitiendo cambiar el nombre del usuario, el mail, la contraseña y ,para los usuarios administradores, cambiar el rol.

RF-04: Procesamiento y registro de compras. El sistema debe registrar las transacciones de compra de entradas, vinculando al usuario autenticado con el sector elegido, almacenando la cantidad de tickets, la fecha/hora y el monto final cobrado en pesos.

RF-05: Autenticación y control de accesos JWT. El sistema debe exigir inicio de sesión mediante (JWT) para realizar compras o acceder al panel estadístico con DjangoModelPermissions (no tiene que ser necesaria la



autenticación para ver las carteleras del festival) para que los perfiles “productor” o “soporte” puedan crear o modificar eventos y sectores.

RF-06: Búsqueda, filtrado y paginación de las carteleras. El sistema debe poder filtrar las noches del festival por rango de fechas, artista y sector de ubicación. Las respuestas de estas búsquedas tienen que darse en forma paginada.

RF-07: Consumo de cotización externa. Con el objetivo de mejorar la accesibilidad de la Fiesta Nacional del Poncho a turistas extranjeros, el sistema debe conectarse automáticamente con el endpoint externo de dolarapi.com para obtener en tiempo real el valor de compra/venta del dólar en Argentina. Utilizando este valor, el sistema calculará y expondrá de manera simultánea el costo de las entradas tanto en pesos argentinos como en dólares. En caso de una pérdida de conexión con dolarapi.com se debe guardar en todo momento el último valor de cotización utilizado como backup.

RF-08: Motor de precios dinámicos. El sistema debe implementar un endpoint que calculará el valor de los tickets combinando el inventario actual con un modelo matemático polinomial determinista. Este modelo evaluará variables temporales (días de antelación) y consumirá la API de YouTube para conseguir las reproducciones mensuales de ese artista y confeccionar el coeficiente de popularidad. El coeficiente resultante determinará el precio.

RF-09: Métricas de eventos a nivel total y específicos que muestren recaudación total por el motor de precios dinámicos, historial de ganancias respecto a ventas y un gráfico específico del nivel de ocupación de los sectores de un evento.

3.2 Requerimientos no funcionales:



RNF-01: Integridad de datos y control de concurrencia. Los serializadores encargados de procesar la compra de entradas deben implementar validaciones avanzadas en el backend que verifiquen, a través de bloqueos transaccionales en la base de datos, que queden asientos reales disponibles en el sector antes de confirmar el pago, evitando la sobreventa de entradas.

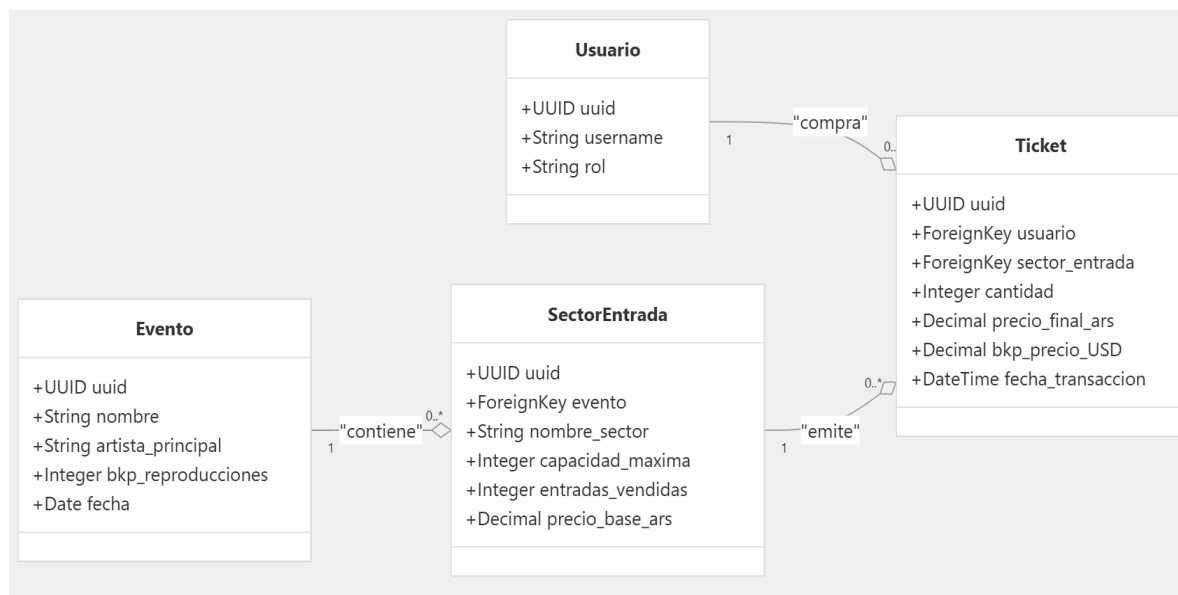
RNF-02: Mitigación de reventas. El serializer de compras debe comprobar que un mismo usuario autenticado no pueda adquirir más de 4 entradas para un mismo sector en una sola transacción.

RNF-03: Tolerancia a Fallos de Servicios Externos (Estrategia de Fallback) El sistema no debe interrumpir el proceso de consulta de la cartelera si los servicios de dolarapi.com o la YouTube Data API se encuentran caídos o fuera de servicio. El backend debe implementar una estrategia de contingencia (caché local) utilizando la última cotización cambiaria registrada y el último índice de popularidad almacenado en la base de datos para asegurar una disponibilidad continua del sistema.

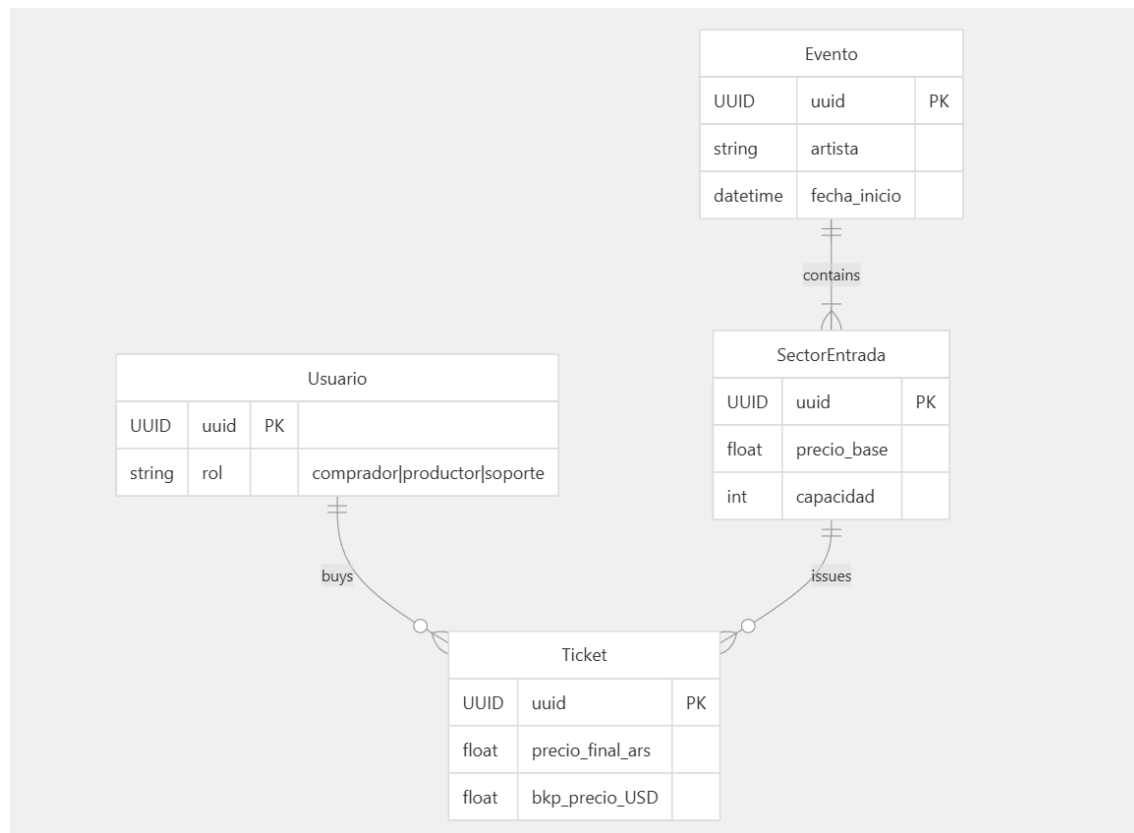
RNF-04: Determinismo matemático. A pesar de que el algoritmo modela un comportamiento de demanda estocástico, la función polinomial de cálculo de precios implementada debe ser estrictamente determinista. Dados los mismos parámetros de entrada (días de antelación, día de la semana e índice de Spotify), el backend debe retornar exactamente el mismo multiplicador de precio.

RNF-05: API no REST. Se debe utilizar como mínimo una API no REST para el sistema.

4. Diagrama UML:



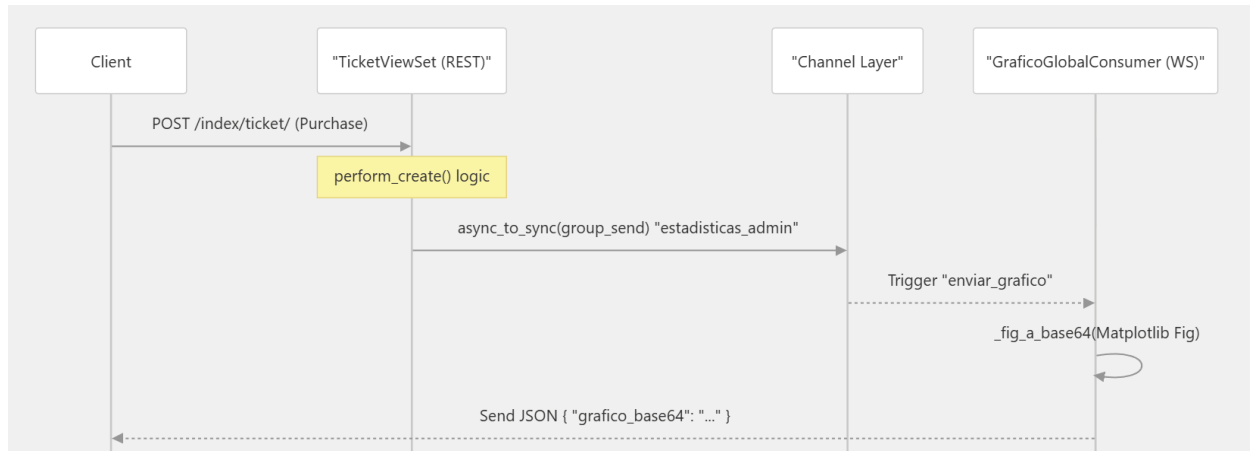
5. Diagrama de entidad relación del sistema:



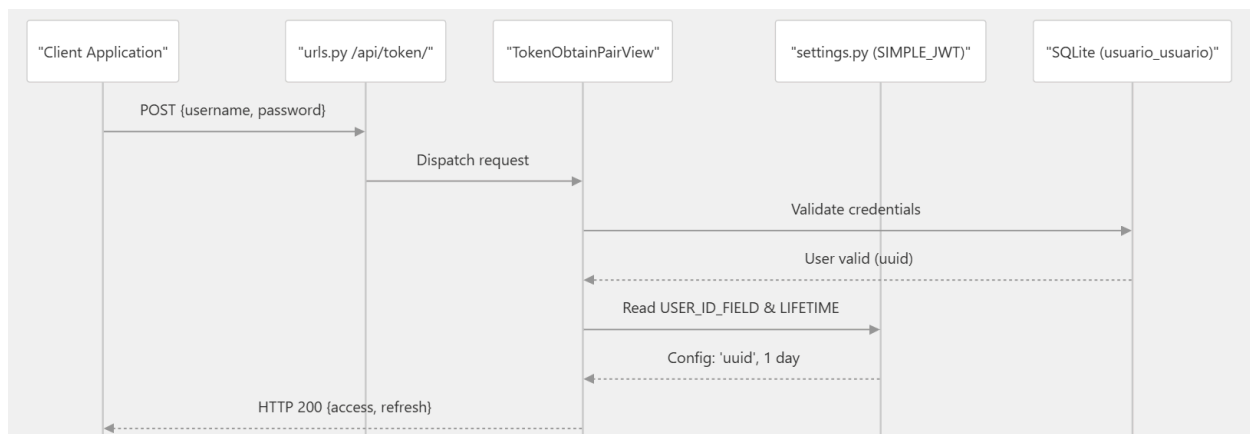
6. Diagramas de secuencia:

El propósito de estos diagramas es ilustrar una de las secuencias de activación del WebSocket, la autenticación de un usuario a través de tokens de JWT y el proceso de funcionamiento del motor de precios dinámicos para la comprensión del funcionamiento interno del sistema.

6.1 DSC del Flujo de WebSocket después de la compra de un Ticket:



6.2 DSC de autenticación basada en tokens JWT:



6.3 DSC de consumo de datos del Motor de precio dinámico:

